

Chicago Bike Trip Data Warehouse & Analytics Solution

Team 5:

By: Derleen Saini (co-leader), Ajinkya Desai (co-leader), Minseok Oh (co-leader)

1. Introduction

- **Data-Driven Revenue Optimization:** Developed an analytics solution that transforms bike trip data into actionable insights to address monotonous pricing models and establish diverse, customer-targeted pricing plans based on user demographics and usage patterns.
- **Supply-Demand Balancing Framework:** Engineered a structured data warehouse with specialized views that identify bike shortages during peak hours and enable demand-driven bike rebalancing to respond to demand fluctuations across stations.
- **Targeted Customer Experience Enhancement:** Implemented demographic and geographical analysis capabilities through a star schema architecture, supporting personalized service offerings for different user groups and optimizing bike availability where and when customers need them most.

2. Data:

https://data.cityofchicago.org/Transportation/Divvy-Trips/fg6s-gzvg/about_data

- Source of Data

The data comes from Divvy, Chicago's bike share system that offers users access to a network of bicycles for short-term rentals through docking stations or a mobile app. The dataset spans 5 years (2013-2017) and contains 21.2 million rows of trip records with 18 columns of attributes.

- Data Attributes

The dataset includes detailed information for each individual bike trip:

- Trip start and end date/time
- Trip start and end station (name, ID, geographic coordinates)
- Bike identification number
- Trip duration in seconds
- Rider type (subscriber, customer)
- Demographic information (gender, birth year) for subscribers

- Important Data Features

1. **Temporal Information:** Complete timestamp data enables time-of-day, day-of-week, seasonal, and multi-year trend analysis.

2. **Geospatial Data:** Origin and destination station coordinates allow for geographic pattern analysis, popular routes identification, and distance calculations.
3. **User Demographics:** Age and gender information for subscribers provides insight into different user segments and their behavior patterns.
4. **Usage Patterns:** Trip duration and rider type categorization help distinguish between casual users and regular commuters, supporting targeted pricing strategies.
5. **System Operations:** The comprehensive station-level data facilitates analysis of station utilization and bike rebalancing needs, addressing supply-demand imbalances.

3. ETL/ELT:

* Data Cleaning Steps Summary: File (1_Data_cleaning.py)

- **Data Loading:** Imported bike trip dataset from CSV file.
- **Initial Analysis:** Performed exploratory data analysis including dataset summary statistics, missing value identification, and data type verification. Analyzed central tendencies for trip duration and identified unique user types.
- **Missing Value Imputation:**
 - Used Random Forest Classification to predict missing gender values based on trip duration and geographic coordinates
 - Applied Random Forest Regression to impute missing birth years using trip duration, gender, and geographic data
 - Filled any remaining missing values with mode (gender) and median (birth year)
- **Geographic Data Cleaning:** Removed rows with missing latitude and longitude values (both origin and destination points).
- **Data Validation:** Verified removal of critical missing values and documented the number of dropped records.
- **Export:** Saved the cleaned dataset as a new CSV file for subsequent analysis stages.

* Transformation and Load Steps Summary: File(2_ETL_process.py)

- **Data Extraction:** Reads cleaned bike trip data from the input CSV file generated by the previous data cleaning step.
- **Transformation Process:**
 - **Standardization:** Converts column names to lowercase with underscores for consistency
 - **Datetime Processing:** Extracts year, month, day, hour, and weekday features from trip start/stop times
 - **Demographic Enhancements:** Calculates rider age from birth year and categorizes into age groups
 - **Categorical Standardization:** Normalizes gender and user type values

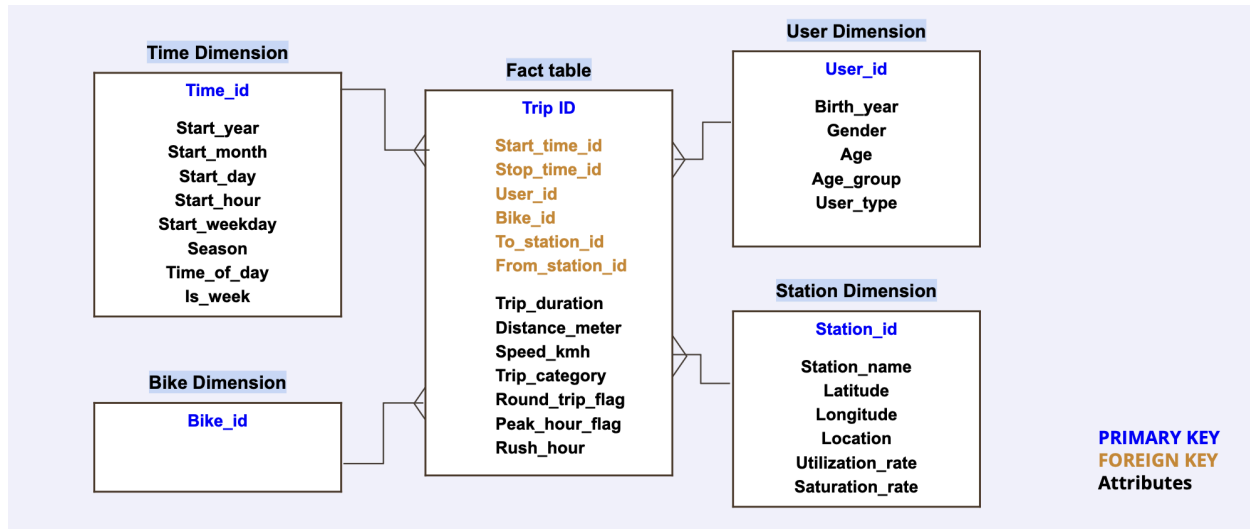
- **Trip Categorization:** Labels trips as Short/Medium/Long/Very Long/Extreme based on duration
- **Temporal Features:** Adds rush hour flags, weekend indicators, time of day classification, and seasonal information
- **Geospatial Analysis:** Calculates trip distance using geodesic formula and categorizes trips by distance
- **Usage Patterns:** Identifies round trips and classifies users by usage intensity
- **Station Analytics:** Computes station utilization and saturation rates based on trip patterns
- **Data Loading:**
 - **Batch Processing:** Segments data by year for efficient database updates
 - **Database Management:** Implements conditional logic for table creation or appending
 - **Incremental Loading:** Removes existing year data before loading new records to prevent duplication
 - **Storage:** Loads transformed data into MySQL database while maintaining data integrity
- **Output:** Saves the fully transformed dataset as 'transformed_bike_data.csv' for analysis and stores it in the 'bike_data_new_half' table in the 'bidw_2025' MySQL database.

4. Star schema

* Star Schema Design for Bike Trip Data Warehouse

Overview

The data warehouse uses a classic star schema architecture with one central fact table (fact_trips2) connected to four dimension tables that provide contextual information for analysis.



Design Rationale

1. **Time Dimension Strategy:**
 - Uses composite surrogate keys (YYYYMMDDHH format) for efficient storage
 - Includes pre-calculated flags (weekend, rush hour) to simplify query complexity
2. **User Dimension Approach:**
 - Simplifies user identification by using trip_id as user_id
 - Includes derived fields like age_group to facilitate demographic reporting
3. **Station Dimension Design:**
 - Combines from/to station data using a UNION ALL approach
 - Stores pre-calculated metrics (utilization/saturation rates) for performance
4. **Fact Table Implementation:**
 - Maintains all necessary dimensional relationships through foreign keys
 - Includes pre-calculated metrics (speed, distance) to avoid redundant calculations
 - Uses flags for common filtering conditions (round trips, peak hours)
5. **Data Validation:**
 - Includes integrity checks for NULL values, outliers, date consistency, and referential integrity
 - Implements business rule validation (e.g., distance vs. location consistency)

Dimension Tables

1. **time_dim2**
 - **Primary key:** time_id (composite key generated from year, month, day, hour)

- **Attributes:** year, month, day, hour, weekday, season, time_of_day, is_weekend, rush_hour
 - **Logic:** Provides temporal context for trip analysis, enabling time-based aggregations and filtering
2. **user_dim2**
- **Primary key:** user_id (derived from trip_id)
 - **Attributes:** birth_year, gender, age, age_group, user_type
 - **Logic:** Captures rider demographics for user behavior analysis and segment-based reporting
3. **station_dim2**
- **Primary key:** station_id
 - **Attributes:** station_name, latitude, longitude, location, utilization_rate, saturation_rate
 - **Logic:** Contains both starting and ending station details with geographic attributes and usage metrics
4. **bike_dim2**
- **Primary key:** bike_id
 - **Attributes:** Only the ID is stored as this is a simple dimension
 - **Logic:** Enables bike-specific analysis and tracking of individual bike usage patterns

Fact Table: fact_trips2

- **Primary key:** trip_id
- **Foreign keys:** start_time_id, stop_time_id, user_id, bike_id, from_station_id, to_station_id
- **Measures:** trip_duration, distance_meters, speed_kmh
- **Flags/Categories:** trip_category, round_trip_flag, peak_hour_flag, rush_hour
- **Logic:** Central table that records each bike trip with all measurable metrics and connects to all dimension tables

Data Mart (Data Warehouse Views): File (4_Data_mart(view_table).sql)

The schema includes several analytical views for simplified querying:

- **view_demand_analysis:** Analyzes trip patterns by time and station pairs
- **view_user_behavior:** Summarizes user behavior by demographic segments
- **view_operational_efficiency:** Measures station-to-station operational metrics
- **view_user_behavior_time:** Tracks user patterns over time periods
- **view_user_behavior_location:** Analyzes user preferences by station location

- **view_user_behavior_patterns:** Examines weekend vs. weekday patterns by demographic
- **view_user_riding_patterns:** Provides individual user riding statistics
- **view_operation_efficiency2:** Comprehensive operational metrics with additional KPIs

```
● CREATE VIEW view_demand_analysis AS
SELECT
    t.start_year,
    t.start_month,
    t.start_day,
    t.start_hour,
    t.start_weekday,
    t.time_of_day,
    t.is_weekend,
    t.rush_hour,
    f.from_station_id,
    fs.station_name AS from_station_name,
    f.to_station_id,
    ts.station_name AS to_station_name,
    COUNT(f.trip_id) AS total_trips,
    AVG(f.distance_meters) AS avg_distance_meters,
    AVG(f.trip_duration) AS avg_trip_duration,
    SUM(CASE WHEN f.rush_hour = 1 THEN 1 ELSE 0 END) AS rush_hour_trips
FROM fact_trips2 f
JOIN time_dim2 t ON f.start_time_id = t.time_id
JOIN station_dim2 fs ON f.from_station_id = fs.station_id
JOIN station_dim2 ts ON f.to_station_id = ts.station_id
GROUP BY t.start_year, t.start_month, t.start_day, t.start_hour,
t.start_weekday, t.time_of_day, t.is_weekend, t.rush_hour,
f.from_station_id, fs.station_name, f.to_station_id, ts.station_name;
```

```
● CREATE OR REPLACE VIEW view_operation_efficiency2 AS
SELECT
    t.start_year,
    t.start_month,
    t.start_day,
    t.start_hour,
    t.start_weekday,
    fs.station_name AS from_station_name,
    ts.station_name AS to_station_name,
    COUNT(f.trip_id) AS total_trips,
    SUM(f.distance_meters) / 1000 AS total_distance_km,
    AVG(f.distance_meters) / 1000 AS avg_distance_km,
    AVG(f.trip_duration / 60) AS avg_duration_minutes,
    SUM(f.trip_duration) / SUM(f.distance_meters / 1000) AS time_per_km_seconds,
    COUNT(DISTINCT f.bike_id) AS unique_bikes_used,
    SUM(CASE WHEN f.round_trip_flag = 1 THEN 1 ELSE 0 END) AS round_trips,
    SUM(CASE WHEN f.round_trip_flag = 1 THEN 1 ELSE 0 END) / COUNT(f.trip_id) * 100 AS round_trip_percentage,
    COUNT(f.trip_id) / COUNT(DISTINCT f.bike_id) AS trips_per_bike,
    SUM(CASE WHEN t.is_weekend = 1 THEN 1 ELSE 0 END) AS weekend_trips,
    SUM(CASE WHEN t.is_weekend = 0 THEN 1 ELSE 0 END) AS weekday_trips,
    SUM(CASE WHEN t.rush_hour = 1 THEN 1 ELSE 0 END) AS rush_hour_trips
FROM fact_trips2 f
JOIN time_dim2 t ON f.start_time_id = t.time_id
JOIN station_dim2 fs ON f.from_station_id = fs.station_id
JOIN station_dim2 ts ON f.to_station_id = ts.station_id
GROUP BY t.start_year, t.start_month, t.start_day, t.start_hour, t.start_weekday,
fs.station_name, ts.station_name;
```

```

-- user
CREATE VIEW view_user_behavior AS
SELECT
    u.user_type,
    u.gender,
    u.age_group,
    COUNT(f.trip_id) AS total_trips,
    AVG(f.distance_meters) AS avg_distance_meters,
    AVG(f.speed_kmh) AS avg_speed_kmh,
    AVG(f.trip_duration) AS avg_trip_duration,
    SUM(CASE WHEN f.round_trip_flag = 1 THEN 1 ELSE 0 END) AS round_trips
FROM fact_trips2 f
JOIN user_dim2 u ON f.user_id = u.user_id
GROUP BY u.user_type, u.gender, u.age_group;

```

```

-- view --
CREATE VIEW view_user_behavior_patterns AS
SELECT
    u.user_type,
    u.gender,
    u.age_group,
    t.is_weekend,
    t.time_of_day,
    COUNT(f.trip_id) AS total_trips,
    AVG(f.distance_meters) AS avg_distance_meters,
    AVG(f.trip_duration) AS avg_trip_duration
FROM fact_trips2 f
JOIN user_dim2 u ON f.user_id = u.user_id
JOIN time_dim2 t ON f.start_time_id = t.time_id
GROUP BY u.user_type, u.gender, u.age_group, t.is_weekend, t.time_of_day;

```

```

CREATE VIEW view_user_behavior_time AS
SELECT
    u.user_type,
    u.gender,
    u.age_group,
    t.start_year,
    t.start_month,
    COUNT(f.trip_id) AS total_trips,
    AVG(f.distance_meters) AS avg_distance_meters,
    AVG(f.speed_kmh) AS avg_speed_kmh,
    AVG(f.trip_duration) AS avg_trip_duration,
    SUM(CASE WHEN f.round_trip_flag = 1 THEN 1 ELSE 0 END) AS round_trips
FROM fact_trips2 f
JOIN user_dim2 u ON f.user_id = u.user_id
JOIN time_dim2 t ON f.start_time_id = t.time_id
GROUP BY u.user_type, u.gender, u.age_group, t.start_year, t.start_month;

```

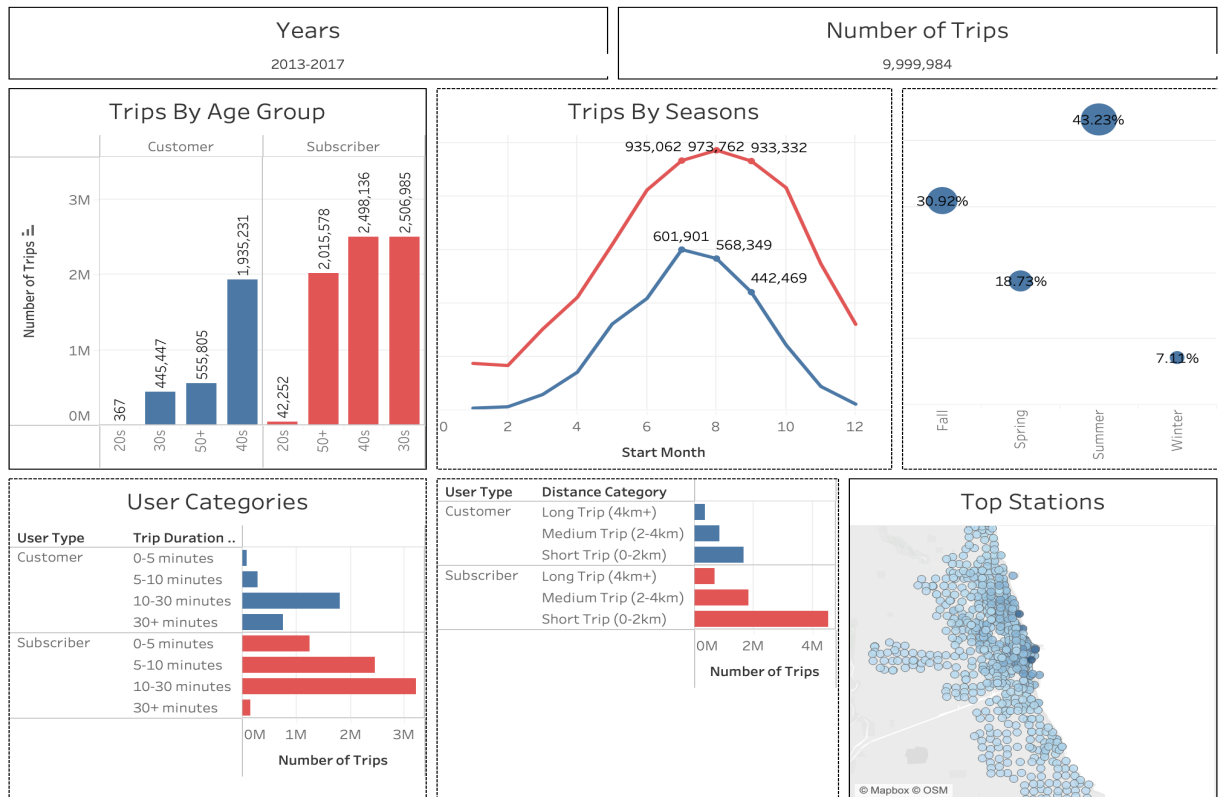
```

-- view --
CREATE VIEW view_user_behavior_location AS
SELECT
    u.user_type,
    u.gender,
    u.age_group,
    fs.station_name AS from_station,
    COUNT(f.trip_id) AS total_trips,
    AVG(f.distance_meters) AS avg_distance_meters,
    AVG(f.trip_duration) AS avg_trip_duration
FROM fact_trips2 f
JOIN user_dim2 u ON f.user_id = u.user_id
JOIN station_dim2 fs ON f.from_station_id = fs.station_id
GROUP BY u.user_type, u.gender, u.age_group, fs.station_name;

```

```
-- view --
CREATE VIEW view_user_riding_patterns AS
SELECT
  u.user_id,
  u.user_type,
  u.gender,
  u.age_group,
  COUNT(f.trip_id) AS total_trips,
  AVG(f.distance_meters) AS avg_distance_meters,
  MAX(f.distance_meters) AS max_distance,
  MIN(f.distance_meters) AS min_distance
FROM fact_trips2 f
JOIN user_dim2 u ON f.user_id = u.user_id
GROUP BY u.user_id, u.user_type, u.gender, u.age_group;
```

- Tableau presentation



Usage Overview

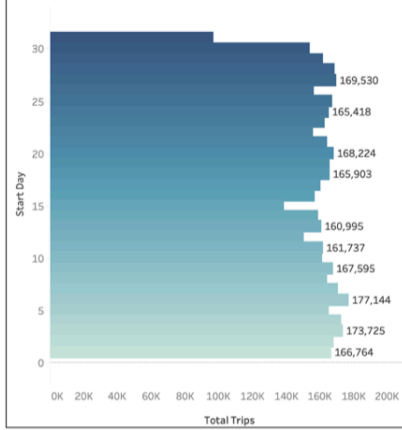
Average Trip Duration(seconds)

1,702.16

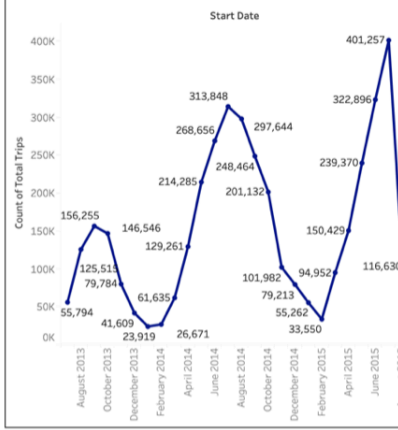
Total Trips

3,989,539

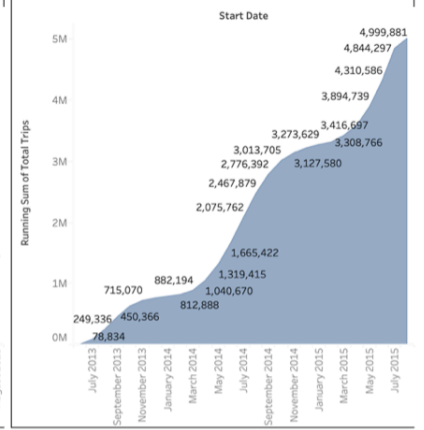
Daily Usage Status



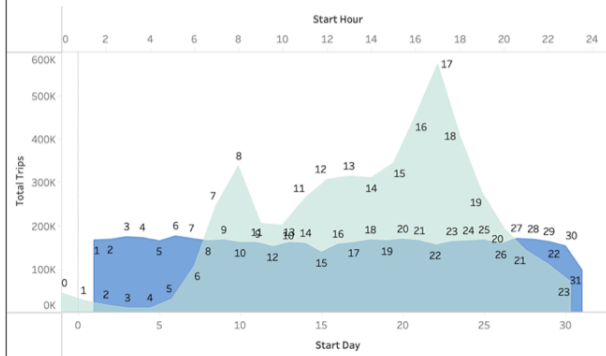
Time Series of Demand



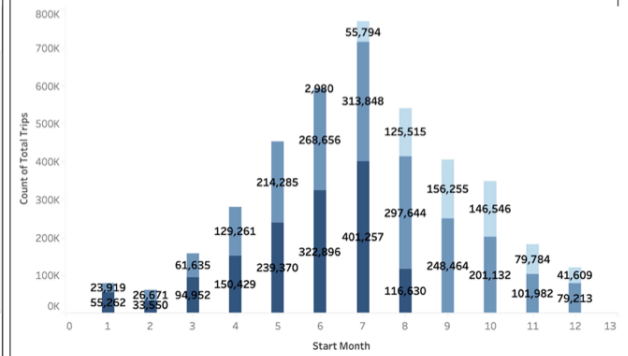
Cumulative Rides Over Time



Usage by Hour and Day



Rides by Month



User Analysis

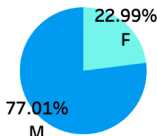
Subscriber vs Customer Ratio

Subscriber: 35.059%
Customer: 64.940%
Unknown: 0.00146%

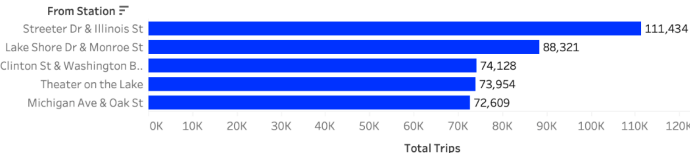
Total Trips by Age Group

Age Group (..	
20s	8,133 0.16%
30s	1,258,194 25.16%
40s	2,416,888 48.34%
50+	1,316,666 26.33%

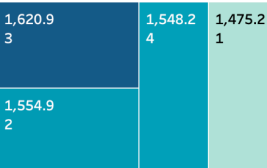
Trips Taken by Male vs Female Users



Top 5 Stations with Highest Trips

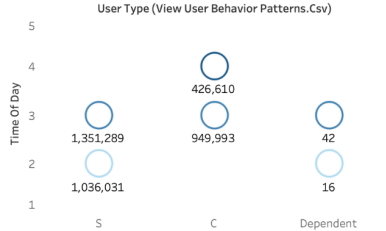


Average Trip Duration for Time of Day

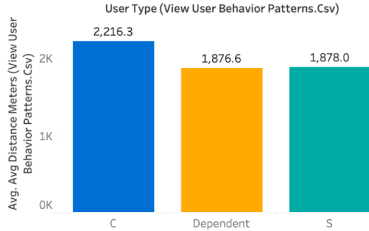


User Type (View User Beh..)
☒ (All)
☒ C
☒ Dependent
☒ S

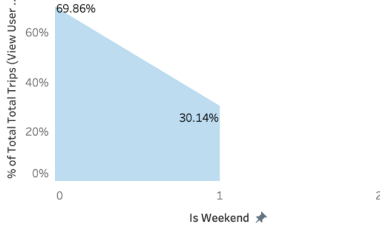
Top 2 Busiest Time of Days



Average Distance by User Type



% of Weekend vs Weekday Trips



Operations Overview

Station Saturation Rate

30.56%

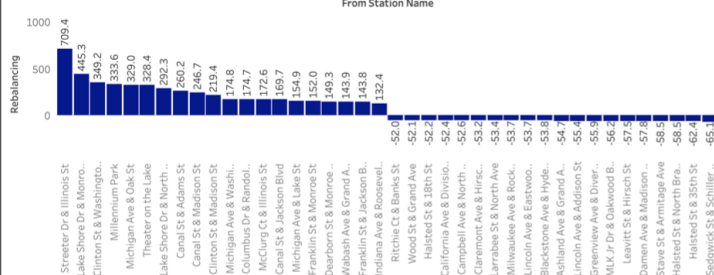
Station Utilization Rate

30.52%

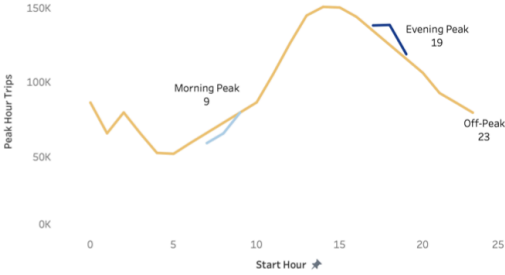
Total Rush Hour Trips

2,046,387

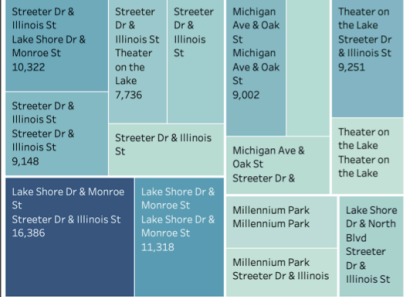
Top / Bottom 20 Stations for Rebalancing



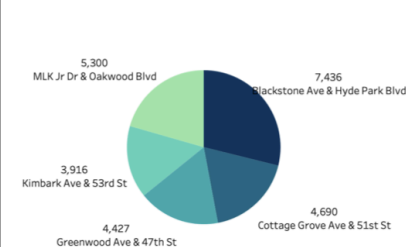
Peak Hour Demand



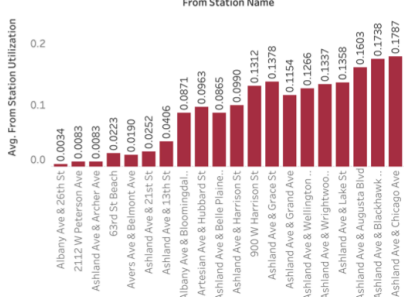
Top 15 Routes



Top 5 Stations by Average Speed



Top 20 Underperforming Stations



- Your recommendations based on 5 OLAP queries
 - 1. Increase Revenue Through Targeted Marketing:** Launch a marketing campaign at the start of the summer to promote annual memberships because riders love to ride bikes during summer season
 - 2. Smart Rebalancing and incentives for off-peak hours** to distribute usage more evenly throughout the day
 - 3. Increase Revenue Through Targeted Pricing & Memberships:**
Subscription Perks & Corporate Plans (\$130/year) during summer to increase user retention
 - 4. Expand stations near high-demand areas**, particularly lakefront locations
 - 5. Targeted Marketing** for Winter months, weekends etc..
 - 6. Increase availability, enhance maintenance, infrastructure and operations** during peak hours
 - 7. How to convert customers to subscribers?**
 - Focus on middle aged users (40s,50s) - age group with the highest usage
 - Offer discounts and loyalty programs
 - Elderly discounted deal
 - Family deal
 - First free month
 - Location based discounts
 - Adding more simplicity for middle aged users
 - Introducing more type of memberships
 - Weekend only membership \$120/year
 - Summer only membership \$130/year
 - Seasonal/quarter memberships
 - Family membership